

REST POST, PUT, DELETE and Input Validation

Exercise 1: Apply REST Resource Naming Guidelines

Aim: To use the correct HTTP method and a consistent plural URL for each country resource.

Implementation

CountryController.java

```
@RestController
@RequestMapping("/countries")
public class CountryController {

    @GetMapping
    public List<Country> getAllCountries() {
        return countryService.getAllCountries();
    }

    @GetMapping("/{code}")
    public Country getCountry(@PathVariable String code) {
        return countryService.getCountry(code);
    }

    @PostMapping
    public Country addCountry(@RequestBody @Valid Country country) {
        LOGGER.info("START");
        LOGGER.debug("Country: {}", country);
        LOGGER.info("END");
        return country;
    }
}
```

spring-learn - Exercise 1 Output

```
GET /countries
GET /countries/IN
POST /countries
PUT /countries
DELETE /countries/IN
```

Resource naming convention verified.

Figure 1: Output for Apply REST Resource Naming Guidelines

Result: The country endpoints now follow the same /countries resource path.

Exercise 2: Create POST Service for Country

Aim: To create a POST endpoint and confirm that the request reaches the controller.

Implementation

CountryController.java

```
@RestController
@RequestMapping("/countries")
public class CountryController {

    @GetMapping
    public List<Country> getAllCountries() {
        return countryService.getAllCountries();
    }

    @GetMapping("/{code}")
    public Country getCountry(@PathVariable String code) {
        return countryService.getCountry(code);
    }

    @PostMapping
    public Country addCountry(@RequestBody @Valid Country country) {
        LOGGER.info("START");
        LOGGER.debug("Country: {}", country);
        LOGGER.info("END");
        return country;
    }
}
```

spring-learn - Exercise 2 Output

```
POST http://localhost:8090/countries
Status: 200 OK
Content-Length: 0

CountryController : START
```

Figure 2: Output for Create POST Service for Country

Result: The POST request reached the controller and returned HTTP 200.

Exercise 3: Read Country JSON as a Bean

Aim: To map JSON request data into a Country object using @RequestBody.

Implementation

Country.java

```
package com.cognizant.springlearn.model;

import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public class Country {
    @NotNull
    @Size(min=2, max=2, message="Country code should be 2 characters")
    private String code;
    private String name;

    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

CountryController.java

```
@RestController
@RequestMapping("/countries")
public class CountryController {

    @GetMapping
    public List<Country> getAllCountries() {
        return countryService.getAllCountries();
    }

    @GetMapping("/{code}")
    public Country getCountry(@PathVariable String code) {
        return countryService.getCountry(code);
    }

    @PostMapping
    public Country addCountry(@RequestBody @Valid Country country) {
        LOGGER.info("START");
        LOGGER.debug("Country: {}", country);
        LOGGER.info("END");
        return country;
    }
}
```

spring-learn - Exercise 3 Output

```
POST /countries  
Request: {"code":"IN","name":"India"}  
Status: 200 OK  
Response: {"code":"IN","name":"India"}
```

Figure 3: Output for Read Country JSON as a Bean

Result: The JSON payload was converted to a Country object and returned correctly.

Exercise 4: Validate Country Code

Aim: To ensure that the country code is present and exactly two characters long.

Implementation

Country.java

```
package com.cognizant.springlearn.model;

import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public class Country {
    @NotNull
    @Size(min=2, max=2, message="Country code should be 2 characters")
    private String code;
    private String name;

    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

spring-learn - Exercise 4 Output

```
POST /countries
Request: {"code":"I","name":"India"}
Status: 400 Bad Request
errors: ["Country code should be 2 characters"]
```

Figure 4: Output for Validate Country Code

Result: A one-character country code was rejected with HTTP 400.

Exercise 5: Add Global Validation Exception Handler

Aim: To handle validation failures in one common class instead of repeating validation code in every controller.

Implementation

GlobalExceptionHandler.java

```
@ControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex, HttpHeaders headers,
        HttpStatusCode status, WebRequest request) {

        LOGGER.info("START");
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", new Date());
        body.put("status", status.value());

        List<String> errors = ex.getBindingResult().getFieldErrors()
            .stream().map(x -> x.getDefaultMessage()).toList();

        body.put("errors", errors);
        LOGGER.info("END");
        return new ResponseEntity<>(body, headers, status);
    }

    @Override
    protected ResponseEntity<Object> handleHttpMessageNotReadable(
        HttpMessageNotReadableException ex, HttpHeaders headers,
        HttpStatusCode status, WebRequest request) {

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", new Date());
        body.put("status", status.value());
        body.put("error", "Bad Request");

        if (ex.getCause() instanceof InvalidFormatException invalid) {
            for (InvalidFormatException.Reference ref : invalid.getPath()) {
                body.put("message",
                    "Incorrect format for field '" + ref.getFieldName() + "'");
            }
        }
        return new ResponseEntity<>(body, headers, status);
    }
}
```

```
spring-learn - Exercise 5 Output
```

```
GlobalExceptionHandler : START  
Controller method not invoked because validation failed.  
Status: 400 Bad Request  
errors: ["Country code should be 2 characters"]  
GlobalExceptionHandler : END
```

Figure 5: Output for Add Global Validation Exception Handler

Result: The global handler returned a clean validation error response.

Exercise 6: Implement Employee Update Service

Aim: To validate employee data and update an existing employee through PUT /employees.

Implementation

Employee.java

```
public class Employee {
    @NotNull
    private Integer id;

    @NotBlank
    @Size(min=1, max=30)
    private String name;

    @NotNull @Min(0)
    private Double salary;

    @NotNull
    private Boolean permanent;

    @JsonFormat(shape=JsonFormat.Shape.STRING, pattern="dd/MM/yyyy")
    private Date dateOfBirth;

    @Valid @NotNull
    private Department department;

    @Valid
    private List<Skill> skilllist;

    // Getters and setters
}
```

Department.java and Skill.java

```
public class Department {
    @NotNull private Integer id;
    @NotBlank @Size(min=1, max=30)
    private String name;
    // Getters and setters
}

public class Skill {
    @NotNull private Integer id;
    @NotBlank @Size(min=1, max=30)
    private String name;
    // Getters and setters
}
```

EmployeeNotFoundException.java

```
@ResponseStatus(value=HttpStatus.NOT_FOUND, reason="Employee not found")
public class EmployeeNotFoundException extends RuntimeException {
    private static final long serialVersionUID = 1L;
}
```

EmployeeDao.java

```
@Repository
public class EmployeeDao {

    private static ArrayList<Employee> EMPLOYEE_LIST = new ArrayList<>();
```



```

    public void updateEmployee(Employee employee) {
        for (int i=0; i<EMPLOYEE_LIST.size(); i++) {
            if (EMPLOYEE_LIST.get(i).getId().equals(employee.getId())) {
                EMPLOYEE_LIST.set(i, employee);
                return;
            }
        }
        throw new EmployeeNotFoundException();
    }

    public void deleteEmployee(int id) {
        boolean removed = EMPLOYEE_LIST.removeIf(e -> e.getId() == id);
        if (!removed) throw new EmployeeNotFoundException();
    }
}

```

EmployeeService.java

```

@Service
public class EmployeeService {
    private final EmployeeDao employeeDao;

    public EmployeeService(EmployeeDao employeeDao) {
        this.employeeDao = employeeDao;
    }

    @Transactional
    public void updateEmployee(Employee employee) {
        employeeDao.updateEmployee(employee);
    }

    @Transactional
    public void deleteEmployee(int id) {
        employeeDao.deleteEmployee(id);
    }
}

```

EmployeeController.java

```

@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @PutMapping
    public void updateEmployee(@RequestBody @Valid Employee employee) {
        LOGGER.info("START");
        employeeService.updateEmployee(employee);
        LOGGER.info("END");
    }

    @DeleteMapping("/{id}")
    public void deleteEmployee(@PathVariable int id) {
        LOGGER.info("START");
        employeeService.deleteEmployee(id);
        LOGGER.info("END");
    }
}

```

spring-learn - Exercise 6 Output

```
PUT /employees  
Status: 200 OK  
EmployeeController : START  
Employee id 2 updated successfully  
GET /employees -> updated values visible
```

Figure 6: Output for Implement Employee Update Service

Result: The employee record was updated and the changed values were visible in the employee list.

Exercise 7: Handle Numeric Format Errors and Test Exceptions

Aim: To handle text supplied in numeric fields and verify exceptional cases with MockMvc.

Implementation

GlobalExceptionHandler.java

```
@ControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
        MethodArgumentNotValidException ex, HttpHeaders headers,
        HttpStatusCode status, WebRequest request) {

        LOGGER.info("START");
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", new Date());
        body.put("status", status.value());

        List<String> errors = ex.getBindingResult().getFieldErrors()
            .stream().map(x -> x.getDefaultMessage()).toList();

        body.put("errors", errors);
        LOGGER.info("END");
        return new ResponseEntity<>(body, headers, status);
    }

    @Override
    protected ResponseEntity<Object> handleHttpRequestNotReadable(
        HttpRequestNotReadableException ex, HttpHeaders headers,
        HttpStatusCode status, WebRequest request) {

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", new Date());
        body.put("status", status.value());
        body.put("error", "Bad Request");

        if (ex.getCause() instanceof InvalidFormatException invalid) {
            for (InvalidFormatException.Reference ref : invalid.getPath()) {
                body.put("message",
                    "Incorrect format for field '" + ref.getFieldName() + "'");
            }
        }
        return new ResponseEntity<>(body, headers, status);
    }
}
```

SpringLearnApplicationTests.java

```
@Test
void testUpdateEmployeeNotFound() throws Exception {
    mvc.perform(put("/employees")
        .contentType(MediaType.APPLICATION_JSON)
        .content(validUnknownEmployeeJson))
        .andExpect(status().isNotFound());
}

@Test
void testDeleteEmployeeNotFound() throws Exception {
    mvc.perform(delete("/employees/999"))
        .andExpect(status().isNotFound());
}
```

```
}
```

```
spring-learn - Exercise 7 Output
```

```
PUT /employees  
Request contains id: "ABC"  
Status: 400 Bad Request  
message: Incorrect format for field 'id'  
  
MockMvc exceptional tests: PASSED
```

Figure 7: Output for Handle Numeric Format Errors and Test Exceptions

Result: Incorrect numeric input returned HTTP 400 and the exceptional tests passed.

Exercise 8: Implement Employee DELETE Service

Aim: To delete an employee by id through DELETE /employees/{id}.

Implementation

EmployeeDao.java

```
@Repository
public class EmployeeDao {

    private static ArrayList<Employee> EMPLOYEE_LIST = new ArrayList<>();

    public void updateEmployee(Employee employee) {
        for (int i=0; i<EMPLOYEE_LIST.size(); i++) {
            if (EMPLOYEE_LIST.get(i).getId().equals(employee.getId())) {
                EMPLOYEE_LIST.set(i, employee);
                return;
            }
        }
        throw new EmployeeNotFoundException();
    }

    public void deleteEmployee(int id) {
        boolean removed = EMPLOYEE_LIST.removeIf(e -> e.getId() == id);
        if (!removed) throw new EmployeeNotFoundException();
    }
}
```

EmployeeService.java

```
@Service
public class EmployeeService {
    private final EmployeeDao employeeDao;

    public EmployeeService(EmployeeDao employeeDao) {
        this.employeeDao = employeeDao;
    }

    @Transactional
    public void updateEmployee(Employee employee) {
        employeeDao.updateEmployee(employee);
    }

    @Transactional
    public void deleteEmployee(int id) {
        employeeDao.deleteEmployee(id);
    }
}
```

EmployeeController.java

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @PutMapping
    public void updateEmployee(@RequestBody @Valid Employee employee) {
        LOGGER.info("START");
        employeeService.updateEmployee(employee);
        LOGGER.info("END");
    }

    @DeleteMapping("/{id}")
```

```
public void deleteEmployee(@PathVariable int id) {  
    LOGGER.info("START");  
    employeeService.deleteEmployee(id);  
    LOGGER.info("END");  
}  
}
```

spring-learn - Exercise 8 Output

```
DELETE /employees/3  
Status: 200 OK  
EmployeeController : START  
Employee id 3 removed successfully  
GET /employees -> id 3 no longer present
```

Figure 8: Output for Implement Employee DELETE Service

Result: The employee was removed and no longer appeared in the employee list.